ASSIGNMENT

Topic: LSTM Practice Questions

Name: Adarsh Singh

Batch: 3

SAP ID: 500119537

Question 1: Numerical (single-cell forward pass)

A scalar LSTM cell receives input x_t = 0.5, with previous hidden state h_(t−1) = 0.2 and previous cell state C_(t−1) = 0.4. The parameters are:

- Forget gate: W_f = 0.7, U_f = 0.3, b_f = 0.1
- Input gate: W_i = 0.6, U_i = 0.4, b_i = 0.0
- Candidate: W_c = 0.9, U_c = 0.2, b_c = 0.0
- Output gate: W_o = 0.5, U_o = 0.5, b_o = 0.1

Compute f_t, i_t, C̃_t, C_t, o_t, and h_t.

Answer:

Forget gate

- z_f = 0.7(0.5) + 0.3(0.2) + 0.1 = 0.35 + 0.06 + 0.1 = 0.51
- f_t = σ(0.51) = 1 / (1 + e^(−0.51)) = 1 / (1 + 0.6005) = **0.6248**

Input gate

- z_i = 0.6(0.5) + 0.4(0.2) + 0 = 0.30 + 0.08 = 0.38
- i_t = σ(0.38) = 1 / (1 + e^(−0.38)) = 1 / (1 + 0.6839) = **0.5939**

Candidate

- z_c = 0.9(0.5) + 0.2(0.2) + 0 = 0.45 + 0.04 = 0.49
- e^0.49 = 1.6323 and e^(−0.49) = 0.6126
- C̃_t = tanh(0.49) = (1.6323 − 0.6126) / (1.6323 + 0.6126) = 1.0197 / 2.2449 = **0.4542**

New cell state

- C_t = f_t · C_(t−1) + i_t · C̃_t
- C_t = 0.6248(0.4) + 0.5939(0.4542) = 0.2499 + 0.2697 = **0.5196**

Output gate

- z_o = 0.5(0.5) + 0.5(0.2) + 0.1 = 0.25 + 0.10 + 0.1 = 0.45
- o_t = σ(0.45) = 1 / (1 + e^(−0.45)) = 1 / (1 + 0.6376) = **0.6106**

Hidden state

- e^0.5196 = 1.6814 and e^(−0.5196) = 0.5948
- tanh(C_t) = tanh(0.5196) = (1.6814 − 0.5948) / (1.6814 + 0.5948) = 1.0866 / 2.2762 = 0.4774
- h_t = o_t · tanh(C_t) = 0.6106(0.4774) = **0.2915**

Question 2: Numerical (parameter count)

An LSTM layer takes 10-dimensional input vectors and has 32 hidden units.

(a) Using the standard formulation with one bias vector per gate (four gates in total), how many trainable parameters does the layer have?

(b) Frameworks such as PyTorch (nn.LSTM) store two bias vectors per gate. How many parameters does the layer have under that convention?

Answer:

Input size = 10, hidden units = 32. Each gate has a weight matrix W (32 × 10), a recurrent matrix U (32 × 32) and a bias vector (32).

(a) One bias vector per gate

- Per gate: 32 × 10 + 32 × 32 + 32 = 320 + 1,024 + 32 = 1,376
- Four gates: 4 × 1,376 = **5,504 parameters**

(b) Two bias vectors per gate (PyTorch)

- Extra bias parameters: 4 × 32 = 128
- Total: 5,504 + 128 = **5,632 parameters**

Question 3: Numerical (vanishing gradient through the cell state)

In a long sequence, the gradient flowing backwards through the cell state from time t to time t − k is scaled approximately by the product of the forget-gate values across those steps, i.e. f^k when the forget gate is constant at f.

(a) Compute the scaling factor for f = 0.9 after k = 10 steps and after k = 50 steps.

(b) For k = 50, compute the scaling factor for f = 0.5 and for f = 0.99.

(c) In two to three sentences, explain what these results mean for learning a dependency 50 time steps back, and how the LSTM design helps compared with a vanilla RNN.

Answer:

Scaling factor = f^k.

(a) f = 0.9

- k = 10: 0.9^{10} = **0.3487**
- k = 50: $0.9^{50} = (0.9^{10})^5 = 0.3487^5$ = **0.0052**

(b) k = 50

- f = 0.5: $0.5^{50} = 1 / 2^{50} = 1 / (1.1259 \times 10^{15}) =$ **8.8818×10^{-16}**
- f = 0.99: $0.99^{50} = e^{(50 \times \ln 0.99)} = e^{(-0.5025)} =$ **0.6050**

(c) Explanation

A forget gate of 0.9 leaves only about 0.5% of the gradient after 50 steps, so a dependency 50 steps back gives almost no learning signal. A vanilla RNN is worse, because its gradient is multiplied at every step by the recurrent weights and the tanh derivative, which usually shrinks it even faster. In an LSTM the cell state is updated additively and the forget gate is learned, so it can stay close to 1 (f = 0.99 keeps about 60%) and let the gradient flow back almost unchanged.

Question 4: Real-world scenario (electricity load forecasting)

A regional utility records hourly electricity load for three full years. Assume 365-day years and no missing data. The team builds an LSTM that uses a 168-hour (one-week) lookback window with three features per hour (load, temperature, holiday flag) to predict the next 24 hours of load.

(a) How many hourly records are in the dataset?

(b) With a stride of 1 hour, how many input/output windows can be generated from the full dataset?

(c) What are the tensor shapes of an input batch of 64 windows and the corresponding output batch?

(d) The first 70% of records are used for training. How many training windows are there, and how many mini-batches (batch size 64, including the final partial batch) does one epoch contain?

(e) If the LSTM layer has a hidden size of 64 and the input has 3 features (single-bias convention), how many trainable parameters does the layer have?

Answer:

(a) Number of hourly records

- 3 years × 365 days × 24 hours = **26,280 records**

(b) Number of windows (stride 1)

- Each window needs 168 input hours + 24 output hours = 192 consecutive hours
- Windows = 26,280 − 192 + 1 = **26,089**

(c) Tensor shapes for a batch of 64

- Input: **(64, 168, 3)** = (batch, time steps, features)
- Output: **(64, 24)** = (batch, forecast hours)

(d) Training windows and mini-batches

- Training records = 0.70 × 26,280 = 18,396
- Training windows = 18,396 − 192 + 1 = **18,205**

Assumption: The training windows are generated entirely within the first 70% of the records; both the 168-hour input window and its 24-hour forecast target must remain inside the training partition.

- Mini-batches = 18,205 / 64 = 284.45, rounded up = **285** (284 full batches of 64 = 18,176 windows, plus 1 partial batch of 29)

(e) LSTM parameters (hidden size 64, input 3, single bias)

- Per gate: 64 × 3 + 64 × 64 + 64 = 192 + 4,096 + 64 = 4,352
- Four gates: 4 × 4,352 = **17,408 parameters**

Question 5: Real-world scenario (predictive maintenance)

A factory monitors a pump with a vibration sensor. During calibration, readings ranged from 20 to 120 mm/s. The team uses an LSTM over the last 50 cycles of sensor data to predict whether the pump will fail within the next cycle.

(a) Apply min-max normalisation to a reading of 95 mm/s.

(b) For one cycle, the LSTM outputs a failure probability of 0.83. With a decision threshold of 0.5, what does the model predict?

(c) On a test set of 1,000 operating cycles, 40 are followed by an actual failure. The model flags 45 cycles as failures, and 30 of those flags are correct. Compute the true positives, false positives, false negatives, and true negatives. Then compute precision, recall, F1-score, and accuracy.

(d) Which metric should the maintenance team prioritise, and why might accuracy be misleading here?

Answer:

(a) Min-max normalisation of 95 mm/s

- $x' = (x - \min) / (\max - \min) = (95 - 20) / (120 - 20) = 75 / 100 =$ **0.75**

(b) Prediction at threshold 0.5

- 0.83 > 0.5, so the model predicts **failure**.

(c) Confusion matrix and metrics (1,000 cycles)

- TP = 30 (correct flags)
- FP = 45 − 30 = **15**
- FN = 40 − 30 = **10**
- TN = 1,000 − 30 − 15 − 10 = **945**

Metric	Working	Value
Precision	TP / (TP + FP) = 30 / 45	0.6667
Recall	TP / (TP + FN) = 30 / 40	0.7500
F1-score	2PR / (P + R) = 2(0.6667)(0.75) / (0.6667 + 0.75)	0.7059
Accuracy	(TP + TN) / 1,000 = 975 / 1,000	0.9750

(d) Which metric to prioritise

The team should prioritise **recall**. A missed failure (false negative) means unplanned downtime and possible damage, which usually costs far more than an unnecessary inspection (false positive). Accuracy is misleading because the classes are imbalanced: only 4% of cycles precede a failure, so a model that always predicts "no failure" would still score 96% accuracy while catching none.